

Project 8: CMD CTF

- Due Apr 9 by 11:59pm
- Points 100
- Submitting an external tool
- Available Apr 1 at 12am - Apr 9 at 11:59pm

This assignment was locked Apr 9 at 11:59pm.

Description and Deliverables

In this project, you will gain hands-on experience leveraging exploits to make a program do unexpected things that were not intended by the programmer. The assignment is structured as a Capture The Flag (CTF), which is a common format in [cybersecurity competitions \(https://defcon.org/html/links/dc-ctf.html\)](https://defcon.org/html/links/dc-ctf.html). You will be given access to a vulnerable program, and your task is to locate **six flags** hidden in the program and its surrounding files. **You are required to find five flags**; the sixth flag is considerably more challenging and is worth bonus points.

To receive full credit for this project, you will turn in a single file:

1. A file named *project8/flags.txt* containing the flags that you will capture from the target program.

The exact format of this deliverable is described in detail below.

Getting Started with Expense Management

Happy Funtime Plastic Co, Inc. uses a piece of software developed in-house for tracking expenses made by employees. This software is run on the command line and allows users to add expenses (with a description and an amount), view their saved expenses, and delete expenses that were entered incorrectly. The program uses an SQLite3 database to store expenses. The command-line syntax of the program is as follows:

```
/home/accounting$ ./expense
```

This program allows users to track their expenses. It has commands for adding, viewing, and deleting expenses. Administrators may add, view, or delete expenses for any user. The program also keeps a log of all changes to facilitate offline auditing.

Usage: `./expense [--admin] <--command> [arguments]`

Commands:

`--help`

Displays this message

`--add <Description> <Amount>`

Adds an expense for the current user

<code>--list</code>	Lists all expenses for the current user
<code>--del <ID></code>	Deletes the current user's expense with the given ID

Additional commands are available to administrators. See the developer docs for more information.

The basic commands available to users are `--add`, `--list`, and `--del`. There are other commands and options available to system administrators; this additional functionality is accessed by passing the `--admin` option to the program. However, access to admin functionality is password-protected.

As security auditors, you have been given access to the *expense* program in a secure environment for testing. To access the *expense* program, SSH into **172.104.24.29**. For example:

```
$ ssh student@172.104.24.29 -p <your assigned port number> when prompted enter your password.
```

The *expense* program is available in the `/home/accounting` directory on my server.

To access the environment, SSH into 172.104.24.29 with your credentials found [here](https://github.com/cy2550repo/cy2550/blob/main/Project8/student_credentials.csv) (https://github.com/cy2550repo/cy2550/blob/main/Project8/student_credentials.csv). Please note that you will log in as the user "student" and you need to specify your specific port number (using `-p`) so you can access your designated container (i.e IF I am logging in as myself, I will do `ssh student@172.104.24.29` (<mailto:student@172.104.24.29>) `-p 21258` and then enter the password when prompted, which is also available in the csv file underneath project8.

The *expense* program is written in C. The (slightly redacted) source code for the program is [available here](https://cbw.sh/static/class/2550/expense-redacted.c) (<https://cbw.sh/static/class/2550/expense-redacted.c>).

Identifying Flags

Your goal is to locate the six flags hidden in the *expense* program, and it's surrounding files. You need to find five flags to get full points; the sixth flag is a bonus. All flags are located somewhere within the `/home/accounting` directory; there is no need to search for other locations in the file system. All flags follow the following format:

```
SECRET_FLAG_<12 characters of random uppercase letter and numbers>
```

For example, a secret flag might look like this:

```
SECRET_FLAG_PLFUDSTG15AS
```

The secret flags can be anywhere within the `/home/accounting` directory: inside the *expense* program, inside databases, inside files, etc. It is 100% feasible to find all six flags; none have been hidden in such a way that it requires wizard-level exploitation skills, or raw amounts of brute force, to locate.

Rules

To make this assignment fun for everyone, we ask that students obey some basic rules of decorum.

- You will achieve various levels of elevated privilege as you progress through this assignment. This will give you the ability to take destructive actions. We ask that you exercise restraint and not destroy any data or files. Please leave things as you found them ;)
- Do not conduct denial of service to the server. Examples of denial of service attacks include but are not limited to: intentionally running large numbers of processes to control CPU and memory resources, intentionally filling the disk to capacity, and flooding the server with network traffic.
- Do not attack your classmates. This includes attempting to crack their passwords, read or alter files in their home directory, or kill their processes.

WARNING: Follow the Rules!

Any student that fails to follow these rules i.e., intentionally attempts to make mischief, will receive a zero on the assignment. That said, we understand that mistakes happen! If you accidentally make changes to project files that you believe may hinder others' abilities to complete the assignment, let us know immediately and we will repair the situation.

If you so desire, attacking the operating system is considered within bounds. If you achieve root, getting all the flags should be trivial. If you do choose to attack the OS, all we ask is that you do not take destructive actions that (1) leave the server in an unusable state; (2) harm your classmates or their files; or (3) generally prevent other students from completing the assignment.

Exploitation Tips

Your user account does not have permission to read all of the files in the */home/accounting* directory. However, the *expense* program has several vulnerabilities that will allow you to learn these secret flags.

Locating all six flags will require you to examine the C source code of the *expense* program, carefully investigate the program's command-line behavior, and investigate the file system around the *expense* program. Several of the exploits necessary to find the flags are related to topics we discussed in class. You can recover the first five flags using inspection tools and SQL injection attacks, but these attacks will require you to think creatively. To recover the 6th flag, in addition to creativity, you may need to carefully construct a special string ;)

The source code for the *expense* program is relatively simple and well-documented. However, online C language tutorials may be useful for students who have never seen code in the language before. If you have questions about the C program and its syntax or the meaning of any line of code and cannot find answers online, please post questions on the Piazza forum.

The *expense* program makes use of an SQLite3 database. Many online resources describe SQL query syntax in general and SQLite3-specific queries in particular.

Unfortunately, the [source code \(https://cbw.sh/static/class/2550/expense-redacted.c\)](https://cbw.sh/static/class/2550/expense-redacted.c) for the *expense* program that we provide has been redacted. Thus, you may need to employ binary analysis tools to inspect and disassemble the contents of compiled programs. Tools like *hexdump*, *strings*, and *objdump* may come in handy; the *man* pages for these programs offer help about their capabilities and syntax. A debugger like *gdb* may be useful for more advanced exploits. As a starting point, look at the code's *authenticate()* and *open_db* functions.

Common programming tools like Python 3, gcc, g++, nasm, and make are available on the server. If additional tools are available via *apt* that you would like installed, send an email or private Piazza message to the course staff.

Note: You do not need buffer overflow to capture the first five flags.

File Format for *flags.txt*

To receive full credit on this assignment, you must turn in a single (ASCII formatted with Unix-style line breaks) text file named *flags.txt* that contains the flags that you have recovered.

You should create a file named *flags.txt* that contains your captured flags. Each flag should be on its own line. For example, your *flags.txt* file might look like the following:

```
SECRET_FLAG_AAAFLAGG
SECRET_FLAG_BBBFLAGG
SECRET_FLAG_CCCFLAGG
SECRET_FLAG_DDDFLAGG
SECRET_FLAG_EEEFLAGG
SECRET_FLAG_FFFFLAGG
```

Your file should contain at least five flags for full points.

Submitting Your Project

To submit your project, do the following:

1. Create a directory `~/cy2550/project8` in the folder corresponding to your git repository.
2. Copy your *flags.txt* file to the `~/cy2550/project8` folder.

3. Add these files to your repository, commit them, and push the committed files to GitHub.
4. Submit your repository to Gradescope.

Grading

This project is broken down as follows (out of 100):

- 20 points each per flag (five required flags)
- 20 *bonus* points for the extra flag

Points can be lost for turning in files in incorrect formats (e.g., not UNIX-line break ASCII), failing to follow specified formatting and naming conventions, or encrypting/signing your file using the wrong keys.

Tips

CTF projects are puzzles: they're best when students are allowed to figure things out for themselves. Thus, we're reluctant to post tips openly that would give away key aspects of the game. That said, if you feel stuck, or like you don't know where to begin, that is okay! Please talk to the professor or the TAs privately, either on Piazza, email, or office hours, and we will be happy to help you get started. We want everyone to enjoy the CTF, so if you feel like you're not making progress and the project is giving you grief, come talk to us!

We will give one hint to get you started: CTFs are sometimes like escape rooms. The solution to one puzzle is often the key that unlocks the next puzzle. Happy hunting!